automatically reflected in the child roles. Consider how you might implement role inheritance in Natter.

8.2.2 Static roles

Now that you've defined how roles map to permissions, you just need to decide how to map users to roles. The most common approach is to statically define which users (or groups) are assigned to which roles. This is the approach taken by most Java EE application servers, which define configuration files to list the users and groups that should be assigned different roles. You can implement the same kind of approach in the Natter API by adding a new table to map users to roles within a social space. Roles in the Natter API are scoped to each social space so that the owner of one social space cannot make changes to another.

> **DEFINITION** When users, groups, or roles are confined to a sub-set of your application this is known as a ***security domain*** or ***realm***.

Listing 8.6 shows the SQL to create a new table to map a user in a social space to a role. Open schema.sql again and add the new table definition to the file. The `user_roles` table, together with the `role_permissions` table, take the place of the old `permissions` table. In the Natter API you'll restrict a user to having just one role within a space, so you can add a primary key constraint on the `space_id` and `user_id` fields. If you wanted to allow more than one role you could leave this out and manually add an index on those fields instead. Don't forget to grant permissions to the Natter API database user.